

§ 17. Цикл `while`

Вспомните!

- Опишите структуру программы на языке Python.
- Что такое условный оператор в Python?
- Что такое операторы и типы данных языка Python?

Вы узнаете:

- об определении понятия «цикл»;
- что такое итерация;
- об алгоритмической структуре цикла с предусловием;
- об особенностях использования оператора `while`.

Цикл – *Цикл* – Loop

Пока – *Θзір* – While

Тело цикла – *Цикл денесі* – Loop body

Итерация – *Итерация* – Iteration

Параметр цикла – *Цикл параметрі* – Loop params

Счетчик – *Санауыш* – Counter

Бесконечный цикл – *Шексіз цикл* – Endless loop

Что такое цикл?

В повседневной жизни мы часто встречаем циклические задачи. К ним можно отнести любой список, планируемый экзамен, ежедневные действия. Действительно, когда мы приходим в магазин, мы покупаем товары из нашего списка и производим покупки до тех пор, пока список не закончится.

Цикл – это последовательность команд, которая выполняется многократно.

В программировании цикл позволяет производить определенные действия несколько раз по определенному условию. Таким образом происходит выполнение последовательности действий несколько раз.

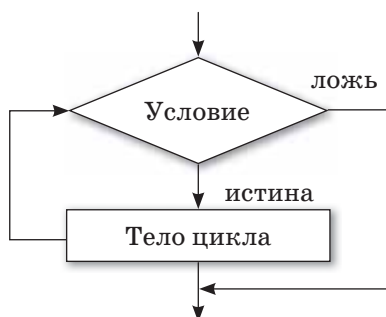
Следует запомнить несколько важных понятий:

- *Тело цикла* – это последовательность кода, который выполняется несколько раз.
- *Итерация* – это выполнение тела цикла.

В Python существует два вида цикла: `for` и `while`.

Что такое цикл `while` в Python?

Цикл, в котором условие проверяется при входе (до следующего шага), называется **циклом с предусловием**. Алгоритмическая структура цикла с предусловием выглядит следующим образом:



Первое условие проверяется во время выполнения цикла с предусловием. Если условие истинно, то выполняется тело цикла, которое повторяется несколько раз до тех пор, пока условие не станет истинным. Если условие ложно, цикл не выполняется и передается команде после цикла управления. Цикл с предусловием используется, когда количество повторений цикла заранее неизвестно.

В Python синтаксис цикла `while` выглядит следующим образом:

```
while условие:
    блок инструкций
```

Переменная, связанная с условием выхода из цикла и меняющая свое значение в цикле, называется **параметром цикла**, или **счетчиком**. Перед выполнением цикла параметру цикла присваивается **начальное значение**.

Общая схема цикла `while` выглядит следующим образом:

```
b = начальное_значение
while b_подходящее_значение:
    оператор
    переход к следующему_b
```

Пример 1. Напишите программу, которая выводит на экран слово «Привет» 5 раз.

В данном примере переменная `k` внутри цикла изменяется от 0 до 5. Такая переменная, значение которой меняется с каждым новым подходом, называется **счетчиком**. Заметим, что после выполнения этого фрагмента значение переменной `k` будет равно 5, поскольку именно при `k = 5` условие `k < 5` перестанет выполняться.

```

k = 0
while k < 5:
    print ('privet')
    k = k + 1

```

Результат выполнения программы:

```

privet
privet
privet
privet
privet

```

Можно организовать цикл другим способом: присвоить счетчику общее количество итераций и при выполнении каждой итерации цикла уменьшать значение счетчика на единицу. В этом случае цикл заканчивается, когда значение счетчика будет равно 0.

```

k = 5
while k > 0:
    print ('privet')
    k = k - 1

```

Результат выполнения программы:

```

privet
privet
privet
privet
privet

```

Пример 2. Вычислите сумму чисел $1 + 2 + 3 + 4 + \dots + n$.

```

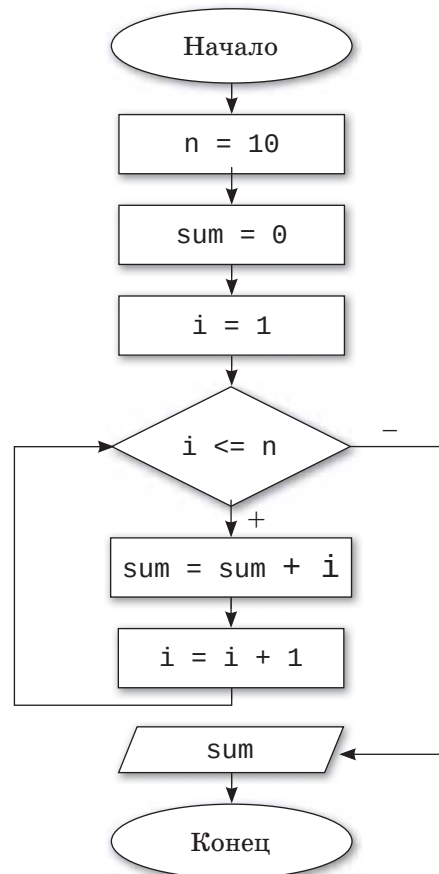
n = 10
sum = 0
i = 1
while i <= n:
    sum = sum + i
    i = i + 1
print ('The sum is', sum)

```

Результат выполнения программы:

The sum is 55

Блок-схема (Пример 2)



Бесконечный цикл

В некоторых случаях, когда неверно задается условие повторения цикла или когда параметр цикла не меняется внутри цикла, происходит заикливание. В таких случаях выход из цикла не выполняется и программа будет выполняться бесконечно. При необходимости можно смоделировать бесконечный цикл. Этот прием используется для того, чтобы программа выполнялась неопределенно долго. Структура бесконечного цикла:

```
k = 1
while k > 0:
    print ('privet')
    k = k + 1
```

В данном случае цикл является бесконечным, потому что условие выполнения цикла всегда будет истинно, в результате вывод на экран слова «privet» будет продолжаться бесконечно.

1

Отвечаем на вопросы

1. Что такое цикл?
2. Какие операторы цикла есть в Python?
3. Как с английского языка переводится слово «while»?
4. Как записывается цикл while?
5. Сколько раз будет выполняться тело цикла, если условие ложно?
6. В каких условиях используется оператор while?

2

Думаем и обсуждаем

1. До каких пор будут выполняться операторы в теле цикла while (x < 100)?
2. Организуйте цикл while, счетчик которого изменяется от 100 до 200 с шагом 2.
3. Укажите, сколько раз выполнится тело цикла с данным заголовком:

```
i = 1
k = -1
while k < 3:
    i = i + 1
    print ('i=', i)
    k = k + 2
```
4. Каким должно быть условие, чтобы тело цикла while ни разу не выполнилось? Почему?

Напишите результат выполнения фрагмента программы.

```
i = 0
while i < 3:
    j = 1
    while j < 3:
        print('i=', i, 'j=', j)
        j = j+1
    i=i+1
```

1. Найдите ошибку в приведенном фрагменте программы.

```
i = 0
while i < 10
    print('i=', i)
```

2. Значение переменных x, y равны $x = 4, y = 6$. Сколько раз выполнится тело цикла в данном фрагменте программы и чему будет равно значения переменных x и y ?

a)

```
while x < y:
    x += 2
```

b)

```
while x < y:
    x += y
```

3. Найдите ошибку в приведенном фрагменте программы. Как можно ее исправить?

```
x = 0
while x < 5:
    print ('SALEM')
```

4. Какой результат выполнения фрагмента программы выйдет на экран?

a)

```
x = 4
while x < 8:
    print (x ** 2, end = '')
    x += 2
```

b)

```
x = 10
```

```
while x > 2:
```

```
    print (2 * x, sep = ' ')
```

```
    x = x - 1
```

5

Выполняем на компьютере



1. Дано натуральное число N . Напишите программу вычисления значения выражения: $(1 - 2) * (1 - 3) * \dots * (1 - N)$.
2. Постройте и запишите в виде программы алгоритм вычисления суммы квадратов 10 произвольных чисел, вводимых с клавиатуры в процессе выполнения программы.
3. Напишите программу, которая выводит на экран таблицу значения функции $y = 3x^2 - 5x - 9$ для x , изменяющегося от -3 до 3 с шагом $-0,5$.
4. Напишите программу вычисления количества цифр в заданном натуральном числе.
5. Выведите на экран все четные числа, которые находятся между числами N и M . Значение чисел N и M задайте сами.
6. Напишите программу вычисления суммы чисел, введенных с клавиатуры, пока сумма не превысит 1000.
7. Напишите программу нахождения суммы четных чисел, находящихся в промежутке от 26 до 88.
8. Введите 14 чисел. Напишите программу, определяющую количество положительных чисел и отрицательных чисел (числа вводятся в одну переменную в цикле).

6

Делимся мыслями

Приведите примеры бесконечного цикла и объясните, почему цикл является бесконечным.

7

Домашнее задание



1. Дана последовательность из n отрицательных чисел, заканчивающаяся положительным числом. Найдите среднее арифметическое всех чисел (без учета положительного числа).
2. Среди чисел 1, 5, 10, 16, 23, ... найдите первое число, больше n . Использование условного оператора не допускается.

§ 18–19. Цикл for

Вспомните!

- Что такое цикл?
- Что такое итерация?
- Как работает оператор while?

Вы узнаете:

- как работает параметрический цикл for;
- о функции range();
- что такое вложенный цикл.

Для – үшін – For

Диапазон – Аралық – Range

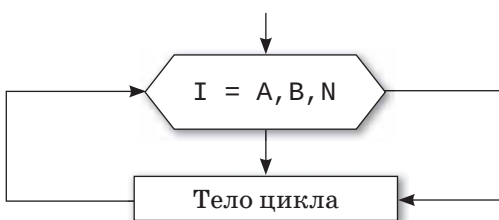
Вложенный цикл – Ішкі цикл –
Nested loop

Что такое цикл for на языке Python?

Допустим, нам нужно организовать цикл, в котором блок операторов будет выполняться несколько раз. Для этого можно использовать еще один вид цикла – **параметрический цикл**. На языке Python параметрический цикл for имеет следующий вид:

```
for <параметр цикла> in <последовательность>:  
    тело цикла
```

Блок-схема цикла for



I – параметр цикла (счетчик),
A – начальное значение параметра,
B – конечное значение параметра,
N – шаг цикла.
Особенность оператора for в Python – использование функции range().

Рассмотрим пример параметрического цикла for: нужно вывести на экран слово «Salem!» 5 раз.

```
for i in range(5):  
    print('Salem!')
```

Результат

Salem!
Salem!
Salem!
Salem!
Salem!

Если набрать и запустить эту программу на компьютере, то слово «Salem!» будет выведено на экран 5 раз. Мы указали

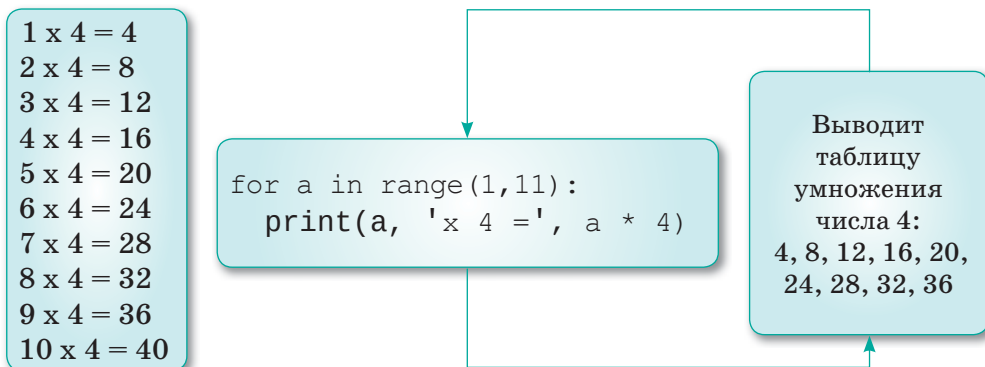
диапазон – число «5» (англ. *range* – «диапазон»). При этом переменная *i* по мере выполнения цикла будет принимать значения 0, 1, 2, 3, 4.

Функция `range()` может состоять из одного, двух или трех аргументов.

range (A) – создает последовательность от 0 до A-1, то есть [0, A-1]. Пример: >>> range (10) [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]	range (A, B) – создает последовательность от A до B, то есть [A, B), B – не входит. Пример: >>> range (5, 10) [5, 6, 7, 8, 9]	range(A, B, N) – создает последовательность от A до B, с шагом N, то есть [A, B), шаг может быть отрицательным. Пример: >>> range (-10, -100, -30) [-10, -40, -70]
---	---	--

Этот вид цикла используется, когда известно количество повторений цикла.

Пример 1. Вывести на экран таблицу умножения числа 4.



Пример 2. С помощью цикла `for` вывести все нечетные числа от 1 до *n*.

```
n = int(input('n ='))  
for j in range(1, n, 2):  
    print (j, end='')
```

Результат:

```
n = 20  
1 3 5 7 9 11 13 15 17 19
```


Пример 3. Вычислите сумму чисел $1 + 2 + 3 + 4 + \dots + n$.

```
n = int(input('vvedite
poslednee chislo'))
s = 0
for i in range(1, n + 1):
    s = s + i
print('summa chisel ot 1
do', n, '=', s)
```

Вложенный цикл

В языке Python есть возможность использования одного цикла внутри другого. Цикл, который встречается внутри другого цикла, называется **вложенным циклом**. Структурно это похоже на выражение if.

Синтаксис вложенных циклов в Python такой:

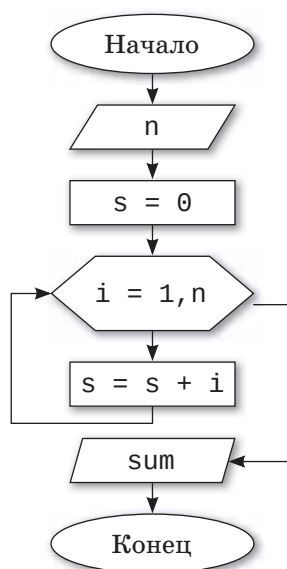
```
переменная for последовательность in:
    переменная for последовательность in:
        оператор (1)
    оператор (2)
```

При выполнении вложенного цикла программа сначала выполняет внешний цикл. При выполнении первой итерации внешнего цикла запускается внутренний цикл. В последующем этот процесс выполняется, пока последовательность не будет завершена или прервана. Одной из особенностей использования вложенных циклов в Python, в том, что один тип цикла можно использовать в другом типе цикла, например, вложить цикл for в цикл while и наоборот.

Пример. Вывести на экран простые числа в диапазоне от 2 до 100.

```
for i in range(2,101):
    for j in range(2,i):
        if (i%j==0):
            break
    else:
        print(i)
```

Блок-схема



Результат:

2	43
3	47
5	53
7	59
11	61
13	67
17	71
19	73
23	79
29	83
31	89
37	97
41	

1

Отвечаем на вопросы

1. Каковы особенности записи и работы цикла `for`?
2. Что такое параметр цикла, начальное и конечное значение цикла?
3. Что такое счетчик цикла?
4. Что такое шаг цикла?
5. Что называют телом цикла с параметром?
6. Каковы особенности порядка выполнения цикла с параметром?
7. Сколько условий требуется для работы оператора цикла с параметром?
8. Что такое вложенный цикл?

2

Думаем и обсуждаем

1. Для чего нужны операторы цикла?
2. Каким будет значение переменной `x` после завершения данного цикла?
`for x in range(100):`
3. Можно ли с помощью оператора `for` организовать цикл, тело которого не будет выполняться? Объясните почему.

1. Данный код выводит на экран квадрат чисел от 10 до 20. Найдите ошибку в данном фрагменте программы:

```
for i in range(10, 20):
    p = pow(i, 2)
print(p)
```

2. Даны натуральные числа от 37 до 87. Выведите на консоль те из них, которые при делении на 7 дают остаток 1, 2 и 5. Найдите ошибку в данном фрагменте программы:

```
for i in range(35, 88):
    if (i % 7 == 1) & (i % 7 == 2) & (i % 7 == 5):
print(i)
```

3. Определите, сколько раз будет выполнен цикл и каков будет результат.

a)

```
for a in range(4):
    print(a)
```

b)

```
for a in range(1, 11):
    print(a)
```

c)

```
for a in range(1, 11, 2):
    print(a)
```

1. Какой результат отобразится на экране после выполнения следующего фрагмента программы?

<pre>S = 0 for i in range(2, 2): S = S + 1 / i print (S)</pre>	<pre>S = 0 for i in range(1, 5): S = S + 1 / i print (S)</pre>	<pre>S = 0 for i in range(2, -2, -1): S = S + i print (S)</pre>
S =	S =	S =

2. Определите, сколько раз будет выполняться тело следующих операторов цикла и чему будет равно значение переменной k .

<pre>k = 0 for i in range(1, k+3): k = k + i print (k)</pre>	<pre>k = 0 for i in range(1, 9): k = k + i*i print (k)</pre>
k=	k=
<pre>k = 0 for i in range(9, 1, -1): k = k + i print (k)</pre>	<pre>k = 0 for i in range(1, 1): k = k + i print (k)</pre>
k=	k=

5

Выполняем на компьютере



1. Напишите программу, которая 7 раз выводит на экран любую строчку из стихотворения.
2. Напишите программу, которая рассчитывает первые 10 степеней трех чисел.
3. Напишите программу вычисления суммы четных чисел от 2 до 30.
4. Напишите программу, которая осуществляет ввод целых чисел с клавиатуры и рассчитывает среднее арифметическое этих чисел.
5. Напишите программу, которая осуществляет ввод целых чисел с клавиатуры и рассчитывает наименьшее этих чисел.
6. Напишите программу, которая осуществляет ввод целых чисел с клавиатуры и рассчитывает наибольшее этих чисел.
7. Напишите программу, которая вычисляет сумму нечетных чисел от 1 до 15.
8. Напишите программу, которая выводит на экран следующие рисунки:

1)	2)
*	*****
**	*****
***	*****
****	*****
*****	*****
*****	***
*****	**
*****	*

6

Делимся мыслями

Можно ли в цикле `for` инициализировать сразу несколько переменных счетчиков? Объясните.

7

Домашнее задание

1. Напишите программу с использованием оператора `for`.
 - a) `i = 1 / 2 + 1 / 4 + 1 / 6 + 1 / 8 + 1 / 10`
`print('i =', i)`
 - b) `f = 1 * 2 * 3 * 4 * 5 * 6 * 7 * 8 * 9 * 10`
`print('f =', f)`
2. Даны вещественные числа x , y . Напишите программу, которая вычисляет произведение и количество чисел в диапазоне от x до y .
3. Напишите программу, которая выводит на экран числовой треугольник следующего вида:


```
1
22
333
4444
55555
666666
7777777
88888888
999999999
10101010101010101010.
```

§ 20. Практикум.

Программирование циклических алгоритмов

Уровень А

1. Один банан стоит 100 тг. Напечатайте таблицу стоимости 2, 3, 30 штук бананов.
2. Считая, что Земля – идеальная сфера с радиусом $R = 6350$ км, определите расстояние до линии горизонта от точки с высотой над Землей, равной 1, 2, ... 10 км.
3. Плотность воздуха убывает с высотой по закону $p = p_0 e^{-hz}$, где p – плотность на высоте h метров, $p_0 = 1,29$ кг/м³, $z = 1,25 \cdot 10^{-4}$. Напечатайте таблицу плотности воздуха в зависимости от высоты, изменяющейся от 1 до 300 метров с шагом 30.
4. Известны данные о мощности двигателей 20 моделей легковых автомобилей. Выясните, есть ли среди них модель, мощность двигателя которой превышает 200 л.с.
5. Известны оценки по предмету «Информатика» 25 учеников класса. Выясните, есть ли среди оценок «Неудовлетворительно».

Уровень В

1. Вычислите сумму $\frac{2}{3} + \frac{3}{4} + \frac{4}{5} + \dots + \frac{10}{11}$.
2. Вычислите сумму $1 + \frac{1}{3} + \frac{1}{3^2} + \dots + \frac{1}{3^8}$. Условный оператор и операцию возведения в степень не использовать.
3. Имеются данные о сумме очков, набранных в чемпионате Казахстана каждой из футбольных команд. Выясните, перечислены ли команды в списке в соответствии с занятыми ими местами в чемпионате.
4. Напечатайте минимальное число, большее 200, которое должно делиться на 17.
5. Выведите на экран все целые числа от 200 до 300, кратным четырем.
6. Известны данные о температуре воздуха в селе Бурабай в течение месяца. Определите, сколько раз температура опустилась ниже 0° С.

Уровень С

1. Арман направлялся к школе, которая находится на расстоянии 1 км от дома. Дойдя до школы, Арман вспомнил, что по прогнозу погоды обещали дождь, а он забыл взять зонт, и поворачивает обратно. Пройдя полпути, он передумал, посчитав, что правильнее пойти в школу. Пройдя $\frac{1}{3}$ км по направлению к школе, он увидел грозовые тучи, и подумал, что все-таки нужно взять зонт. На этот раз, прежде чем изменить свое решение, он проходит $\frac{1}{4}$ км. Так он продолжает ходить, и после n попыток, пройдя $\frac{1}{n}$, снова меняет свое решение.

Определите, на каком расстоянии от дома будет находиться Арман после 50-й попытки (если допустить, что такое возможно), а также посчитайте общий путь.

2. Идет уборка урожая в Алматинской, Жамбылской, Кызылординской, Туркестанской, Восточно-Казахстанской, Акмолинской, Костанайской и Северо-Казахстанской областях. Заданы площади, засеянные пшеницей (в гектарах), и средняя урожайность (в центнерах с гектара) в каждой области.

Определите количество пшеницы, собранной по республике, и среднюю урожайность.

3. Известны расстояния от Нур-Султана до нескольких городов. Найдите расстояние от Нур-Султана до самого удаленного от него города из представленных в списке городов.
4. Дано натуральное число. Выясните, является ли оно простым (простым называется натуральное число, не имеющее других делителей, кроме единицы и самого себя). Оператор цикла с параметром не использовать.

§ 21. Управление циклом `continue`

Вспомните!

- Какую функцию выполняет оператор `while`?
- Какую функцию выполняет оператор `for`?

Вы узнаете:

- об использовании оператора `continue` в работе с циклом.

Продолжение – Жалғастыру – `Continue`

Условие – Шарт – `Condition`

Текущая итерация – Ағымдағы итерация – `Current iteration`

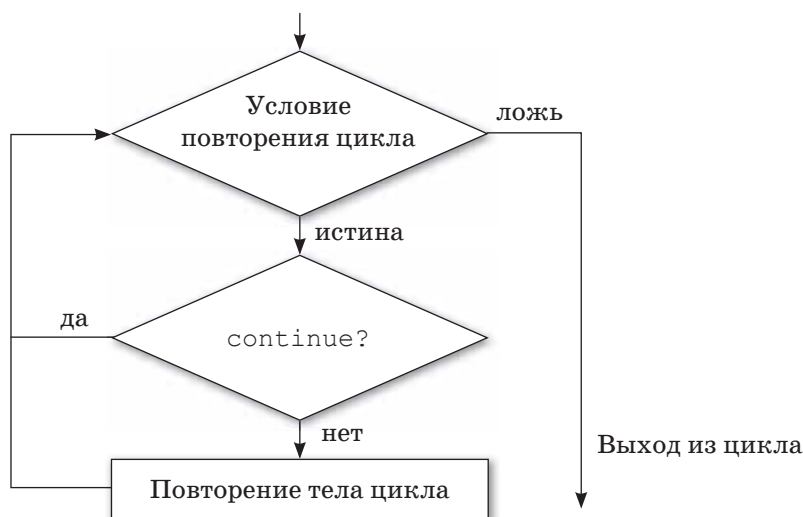
Тело цикла выполняется, пока условие истинно, но в некоторых случаях можно закончить текущую итерацию или цикл, полностью не проверяя условие. В этом случае можно использовать операторы `break` и `continue`. В Python оператор `continue` позволяет изменить порядок выполнения цикла.

Оператор `continue` переходит к следующему шагу, не выполняя часть кода для текущей итерации цикла. Цикл не заканчивается, но продолжается следующая итерация.

Синтаксис оператора `continue` в Python:

`continue`

Блок-схема оператора `continue`:



Работа оператора `continue` в цикле `for` и `while`:

параметр цикла <code>for</code> последовательность <code>in</code> :	<code>while <условие>:</code>
→ тело цикла	→ тело цикла
<code>if условие:</code>	<code>if условие:</code>
<code>continue</code>	<code>continue</code>
команды внутри цикла	команды внутри цикла
команды вне цикла	команды вне цикла

Пример. Напишите программу, которая выводит на экран четные числа в диапазоне от 0 до 100:

```
for i in range(100):
    if not i % 2 == 0:
        continue
    print(i)
```

Функция `range` дает нам диапазон чисел от 0 до 100. Вместо того, чтобы печатать четные числа, мы продолжаем следующую итерацию, если число нечетное. Оператор `continue` переходит к следующей итерации цикла без выполнения следующего кода.

Результат:

0	10	20	30
2	12	22	32
4	14	24	34
6	16	26	36
8	18	28	38

1

Отвечаем на вопросы

1. Каковы особенности записи и выполнения оператора `continue`?
2. Каково предназначение оператора `continue` в цикле `for`?
3. Каково предназначение оператора `continue` в цикле `while`?

2

Думаем и обсуждаем

1. Для чего нужен оператор `continue`?

2. Какими будут результаты выполнения следующих фрагментов программы?

a) `for i in range(9):`

`if i == 3:`

`continue`

`print (i)`

b) `i = 0`

`while i < 9:`

`i += 1`

`if i == 3:`

`continue`

`print (i)`

c) `for l in 'table':`

`if l == 't':`

`continue`

`print (l)`

3

Анализируем и сравниваем

1. Программа выводит все числа от 0 до 9, кроме числа 5. Найдите ошибку в приведенном фрагменте программы. Каким будет результат выполнения программы?

```
var = 10
```

```
while var > 0:
```

```
    var = var -1
```

```
    if var = 5:
```

```
        continue
```

```
print ('Значение текущей переменной:', var )
```

```
print ('Пока!=')
```

2. Данный код программы выводит числа от 1 до 10, кроме числа 6. Найдите ошибку в приведенном фрагменте программы и определите результат выполнения программы.

```
for i in range(1, 10)
```

```
    if i == 6:
```

```
        continue
```

```
    else:
```

```
        print(i, end = ' ')
```

Ответьте на вопросы, выполните задания.

a)

```
i=0
while i < 9:
    i += 1
    if i == 3:
        continue
    print(i)
```

1. Нарисуйте блок-схему программы.
2. Что выполняется в коде и каким будет результат выполнения программы?
3. Напишите программу с использованием оператора `for`.

b)

```
for i in ['milk',
         'bread', 'cake',
         'cherry']:
    for j in i:
        if i == 'cake':
            continue
    print(i)
```

1. Нарисуйте блок-схему программы.
2. Что выполняется в коде и каким будет результат выполнения программы?
3. Напишите программу с использованием оператора `while`.



1. Напишите программу, которая выводит на экран числа от 0 до 6, кроме чисел 3 и 6.
2. Введите с клавиатуры словосочетание. Напишите программу, которая выводит на экран только гласные буквы.
3. Дана последовательность «abcdef». Напишите программу, которая выводит на экран все буквы, кроме *a* и *d*.
4. Дано слово. Напишите программу, которая выводит на экран все буквы, кроме *L*.

5. Наберите на компьютере следующий фрагмент программы и определите результат выполнения программы.

```
k = 5
while k > 0:
    k = k - 1
    if k == 2:
        continue
    print(n)
print('Конец цикла')
```

6

Делимся мыслями

Можно ли заменить оператор `continue` другим оператором в программе для перехода к следующей итерации цикла?

7

Домашнее задание

1. Напишите программу, которая вычисляет сумму положительных чисел из данной последовательности.
2. В 1-й ферме 1000 овец. Каждый день количество овец этой фермы увеличивается на 1%. Если в конце месяца количество овец увеличится на 50000, то 10% овец переведут на 2-ю ферму. Через сколько времени количество овец на 2-й ферме превысит 35000? (Нужно учитывать, что в одном месяце 30 рабочих дней).

§ 22. Управление циклом break

Вспомните!

- Каково предназначение оператора `continue`?
- Каково предназначение оператора `continue` в цикле `for`?
- Какую функцию выполняет оператор `while`?

Вы узнаете:

- об использовании оператора `break` в работе с циклом.

Прерывание – *Үзу* – Break

Внутренний цикл – *Ішкі цикл* – Inner loop

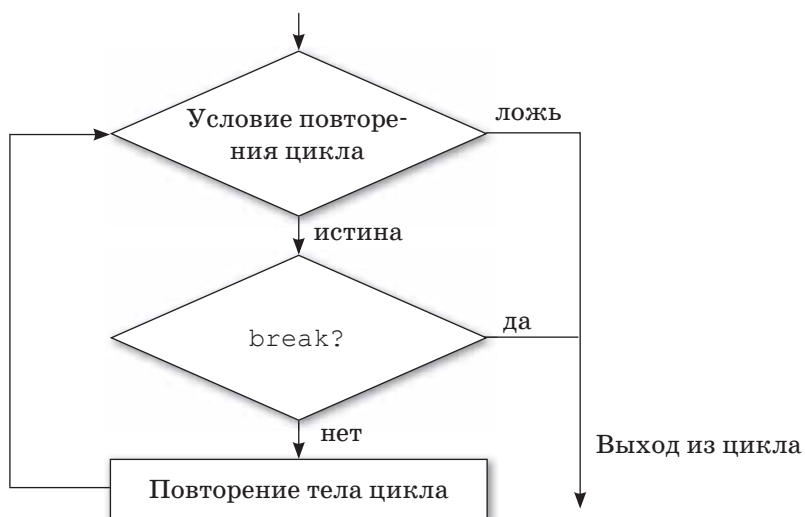
Выход из цикла – *Циклдан шығу* – Exiting the loop

Оператор `break` языка Python используется для выхода из цикла, так как, если в процессе выполнения кода программа встречает оператор `break`, то он сразу заканчивает выполнение цикла и выходит из цикла. Управление программой переходит к оператору, который идет после тела цикла. Если оператор `break` идет во вложенном цикле, то заканчивает выполнение внутреннего цикла.

Синтаксис оператора цикла `break` в Python:

break

Блок-схема оператора `break`:



Работа оператора break в циклах for и while.

параметр цикла for последовательность in:	while <условие>:
тело цикла	тело цикла
if условие:	if условие:
break	break
команды в теле цикла	команды в теле цикла
команды вне цикла	команды вне цикла

Пример 1. Код программы, которая завершает выполнение программы при встрече с числом 5 в последовательности чисел от 1 до 10.

```
n = 0
for n in range(10):
    n = n + 1
    if n == 5:
        break
    print('Число' + str(n))
print('Выход из цикла')
```

Результат:

```
Число 1
Число 2
Число 3
Число 4
Выход из цикла
```

Пример 2. Код программы, завершающий выполнение цикла при встрече числа 5 в данной последовательности чисел.

```
k = 10
while k > 0:
    print ('Текущее значение переменной:', k)
    k = k - 1
    if k == 5:
        break
print ('Пока!')
```

Результат:

Текущее значение переменной: 10
Текущее значение переменной: 9
Текущее значение переменной: 8
Текущее значение переменной: 7
Текущее значение переменной: 6
Пока!

1

Отвечаем на вопросы

1. Как записывается оператор `break` и в чем его особенность?
2. Каково предназначение оператора `break` в цикле `for`?
3. Каково предназначение оператора `break` в цикле `while`?
4. В чем различие операторов `break` и `continue`?

2

Думаем и обсуждаем

1. Для чего предназначен оператор `break`?
2. Какими будут результаты выполнения фрагментов программы?

a)

```
for letter in 'Стол':  
    if letter == 'т':  
        break  
    print (letter)
```

b)

```
for k in 'Python':  
    if k == 't':  
        break  
    print ('Current Letter: ', k)
```

c)

```
var = 10  
while var > 0:  
    print ('Current variable value:', var)  
    var = var - 1  
    if var == 5:  
        break  
    print ('Good bye!')
```

3

Анализируем и сравниваем

Объясните работу операторов `break` и `continue` на операторах цикла `while` и `for`.

4

Выполняем в тетради

1. Дана последовательность чисел от 0 до 10. Напишите программу, которая вычисляет сумму чисел до 5.
2. Дано натуральное число `N`. Напишите программу, которая определяет, является ли число простым.
3. Определите условие задачи и нарисуйте блок-схему.

```
n = 5
while n > 0:
    n = n - 1
    if n == 2:
        break
    print(n)
print('Цикл завершен')
```

5

Выполняем на компьютере



Наберите на компьютере следующие фрагменты программы и определите результат.

```
a)
for s in 'Python':
    if s == 't':
        break
    print(s)
print('Выход из цикла')

b)
for char in 'PYTHON STRING':
    if char == ' ':
        break
    print(char, end = ' ')
    if char == 'O':
        continue
```


Сколько раз выполнится цикл и каким будет значение переменных k , l , s ?

```
k = 1
l = 1
while True:
    k += 2
    l *= 2
    if l > 10: break
    s = k + l
```

Ответьте на вопросы, выполните задания.

a) <pre>for n in 'abcdef': if n == 'c' or n == 'd': break print('letter :', n)</pre>	1. Нарисуйте блок-схему программы. 2. Что выполняется в коде и каким будет результат выполнения программы? 3. Напишите программу с использованием оператора <code>while</code>.
b) <pre>n = 1 while True: print(n) n += 1 if n == 5: break print('After Break')</pre>	1. Нарисуйте блок-схему программы. 2. Что выполняется в коде и каким будет результат выполнения программы? 3. Напишите программу с использованием оператора <code>for</code>.
c) <pre>for char in 'PYTHON STRING': if char == ' ': break print(char, end=' ') if char == 'O': continue</pre>	1. Нарисуйте блок-схему программы. 2. Что выполняется в коде и каким будет результат выполнения программы?

§ 23. Управление циклом else

Вспомните!

- Какую функцию выполняют операторы `for`, `while`?
- Какую функцию выполняют операторы `continue`, `break`?

Вы узнаете:

- об особенностях использования оператора `else` в конструкциях `for/while`.

Блок – Блок – Block

Операторы управления –
Басқару операторлары –
Control statements

Иначе – Әйтпесе – Else

Во многих языках программирования (C / C ++, Java и др.) использование оператора `else` ограничено условным оператором `if`. Но в языке Python оператор `else` можно использовать в операторе цикла.

Как пользоваться оператором `else` в цикле `for`?

В цикле `for` оператор `else` можно написать после тела цикла, если условие выполнения цикла будет ложным, после выхода из цикла выполняется следующий блок операторов.

Пример.

```
for k in range(4):  
    print(k)  
else:  
    print('конец')
```

Результат:

```
0  
1  
2  
3  
конец
```

Оператор `break`

Как известно, оператор `break` позволяет прервать выполнение цикла. Если в цикле `for` используется оператор `break`, то блок после оператора `else` не выполняется.

Базовая структура цикла `for / else`:

if условие:

оператор 1

break

else:

оператор 2

Пример. Напишите программу, которая выводит на экран последовательность чисел, если число равно 2, выполнение цикла останавливается.

```
for k in range(4):  
    print(k)  
    if k == 2:  
        break  
else:  
    print('этот текст не выводится на экран')
```

Результат:

0

1

2

Оператор `continue`

Если в теле цикла встречается оператор `continue`, то текущая итерация цикла не выполняется, но это никак не влияет на работу оператора `else`.

Пример.

```
for el in range(4):  
    if el == 2:  
        continue  
    print(el)  
else:  
    print('конец')
```

Результат:

0

1

3

конец

Как пользоваться оператором *else* в цикле *while*?

Структура *while/else* позволяет выполнить цикл, пока условие выполнения цикла истинно.

Пример. Написать программу, которая выводит на экран слово «выполняем», пока условие повторения цикла истинно, иначе выводит на экран слово «завершаем».

```
k = 0
while k < 5:
    print('выполняем')
    k = k + 1
else:
    print('завершаем')
```

Результат:

```
выполняем
выполняем
выполняем
выполняем
выполняем
завершаем
```

Для выполнения цикла выражение, идущее после *while*, проверяется на истинность:

- если выражение истинно, то выполняется тело цикла;
- если выражение ложно, то тело цикла не выполняется, но выполняется блок *else*, если он задан.

Оператор *break*

Если в теле цикла встречается оператор *break*, то цикл завершается, а блок *else* не выполняется.

Пример.

```
do = True
while do:
    print('выполняем')
    break
do = False
print('выполняем')
else:
    print('завершаем')
```

Результат:

выполняем

Оператор *continue*

Если в теле цикла используется оператор `continue`, то выполнение текущего шага пропускается и проверяется условие выполнения следующего шага.

Пример.

```
do = True
while do:
    print('выполняем')
    do = False
    continue
    print('выполняем')
else:
    print('завершаем')
```

Результат:

выполняем

завершаем

Блок `else` выполняется после операторов `for/while`, если цикл не будет завершен оператором `break`.

1

Отвечаем на вопросы

1. Как записывается оператор `else`? Какова его особенность?
2. Каково предназначение оператора `else` в цикле `for`?
3. Каково предназначение оператора `else` в цикле `while`?

2

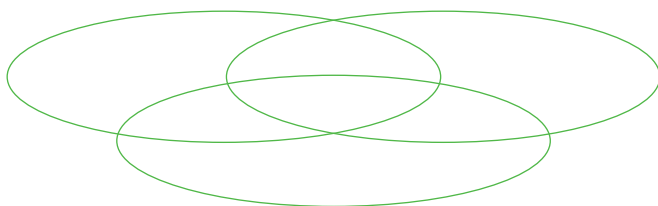
Думаем и обсуждаем

Для чего предназначен оператор `else`?

3

Анализируем и сравниваем

Используя диаграмму Венна, скажите, в чем отличие оператора `else` от операторов `break` и `continue`?



4

Выполняем в тетради

Ответьте на вопросы, выполните задания:

```
a)
i = 1
while i <= 10:
    print(i)
    i += 1
else:
    print('Конец цикла, i = ', i)
```

1. Нарисуйте блок-схему программы.
2. Что выполняется в коде и каким будет результат выполнения программы?

```
b)
for k in range(5):
    print(k)
else:
    print('конец')
```

1. Нарисуйте блок-схему программы.
2. Что выполняется в коде и каким будет результат выполнения программы?

5

Выполняем на компьютере



1. Введите на компьютере заданный программный код и определите результат:

```
n = int(input())
for i in range(n):
    if i % 2 == 0:
        print('найдено четное число')
        break
else:
    print('не найдено четное число')
print('конец программы')
```

2. Определите условие задачи заданного программного кода и каким будет результат выполнения программы.

```
i = 0
while i < 5:
    if i == 3:
        break
    else:
        print(i)
        i += 1
else:
    print('Конец')
```

6

Делимся мыслями

В чем отличие использования оператора `else` в конструкции `for/while` и в конструкции условного оператора `if`?

7

Домашнее задание 

1. Напишите программу, которая выводит на экран последовательность чисел до заданного числа n с использованием инструкции `for/else, while/else`.
2. Напишите программу, которая выводит на экран последовательность чисел до заданного числа n с использованием инструкции `for/else, while/else`. При $n = 3$ остановить вывод на печать.

Творческое задание

Составьте ребусы, в которых зашифрованы операторы цикла.

§ 24. Трассировка алгоритма

Вспомните!

- Что такое цикл с предусловием?
- Как переводится слово «while»?
- Как записывается оператор с предусловием?
- Сколько раз будет выполняться тело цикла, если условие ложно?
- В каких случаях целесообразно использовать оператор с предусловием?

Вы узнаете:

- что такое трассировка;
- что такое ручная трассировка;
- что такое трассировочная таблица.

Трассировка – *Трассировка* – Tracing

Ручная трассировка – *Қолмен жасалатын трассировка* – Manual tracing

Таблица трассировки – *Трассировка кестесі* – Trace table

Для того, чтобы проверить правильность алгоритма, не обязательно переводить его на язык программирования и выполнять тесты на компьютере. Протестировать алгоритм может и человек, путем трассировки.

Трассировка – это пошаговое выполнение программы; действие используется для проверки работоспособности, поиска ошибок в алгоритме и т.д.

Выполняя **ручную трассировку**, человек моделирует работу процессора, исполняя каждую команду алгоритма и занося результаты выполнения команд в трассировочную таблицу. Ручная трассировка производится в ходе заполнения трассировочной таблицы. **Трассировочная таблица** – модель работы процессора при исполнении алгоритма.

Пример 1. Построим трассировочную таблицу для алгоритма «Вычисление суммы чисел от 1 до 5».

Оператор	Условие	N	S	Примечание
S := 0			0	
for N in range(1, 6)	да	1		
S := S + N			1	0 + 1 = 1
for N in range(1, 6)	да	2		
S := S + N			3	1 + 2 = 3
for N in range(1, 6)	да	3		

Оператор	Условие	N	S	Примечание
S := S + N			6	3 + 3 = 6
for N in range(1, 6)	да	4		
S := S + N			10	6 + 4 = 10
for N in range(1, 6)	да	5		
S := S + N			15	10 + 5 = 15
print ('Сумма чисел', S)		???		На экране: Сумма чисел = 15

Для операторов, выполняющих проверку условий (if, for и т. п.), в столбце «Условие» принято указывать результат проверки. В данном случае в цикле for проверяется условие продолжения цикла.

Символы «???» означают, что значение счетчика цикла неизвестно при выходе из цикла.

Метод трассировки помогает при отладке программы, когда программа выдает не тот результат, который должна выдать по замыслу разработчика. Осуществляя пошаговую трассировку, мы вникаем в логику работы программы и на каждом шаге проверяем, правильны ли были наши рассуждения при ее написании. Таким образом, алгоритм в совокупности с трассировочной таблицей полностью моделирует процесс обработки информации, происходящий в компьютере.

Пример 2. Рассчитаем значение переменной *b* по данной блок-схеме.

Шаг	Команда алгоритма	Переменные		Условие
		<i>a</i>	<i>b</i>	
				<i>a</i> = 8
1	<i>a</i> =1	1		
2	<i>b</i> =1		1	
3	<i>a</i> = <i>a</i> *2	2	1	
4	<i>b</i> = <i>b</i> + <i>a</i>	2	3	
5	<i>a</i> =8			2=8 (нет)
6	<i>a</i> = <i>a</i> *2	4	3	
7	<i>b</i> = <i>b</i> + <i>a</i>	4	7	
8	<i>a</i> =4			4=8 (нет)
9	<i>a</i> = <i>a</i> *2	8	7	
10	<i>b</i> = <i>b</i> + <i>a</i>	8	15	
11	<i>a</i> =8			8=8 (да)
12	Вывод <i>b</i>		15	

